```
import os
import json
import time
from datetime import datetime, timedelta
import pytz
import asyncio
from telegram import Bot
from telegram.error import TelegramError
import aiohttp
import pandas as pd
import numpy as np
from collections import deque

# Configuration
TELEGRAM_BOT_TOKEN = "8987349290:AAHM8XxdqPz1W1x9u3k6plYZYq9EDeVbGDw"
TELEGRAM_USER_ID = 7919725795
PAIRS = ["EURUSD", "GBPUSD", "USDJPY"]
TIMEFRAMES = ["1M", "5M", "10M"]
SIGNAL_TIME_START = "18:30"  # 6:30 PM IST
SIGNAL_TIME_END = "20:30"    # 8:30 PM IST
MIN_ACCURACY = 90  # Only 90%+ signals
IST = pytz.timezone('Asia/Kolkata')

# Store candle data
candle_data = {pair: {tf: deque(maxlen=200) for tf in TIMEFRAMES} for pair in PAI
signal_count = {pair: 0 for pair in PAIRS}
last_signals = {}

class ForexDataFetcher:
    """Fetch real forex data"""

    @staticmethod
    async def get_price_data(pair, timeframe="1m"):
        """Get real-time forex data"""
        try:
            # Using yfinance for real forex data
            import yfinance as yf

            ticker = f"{pair}=X"

            # Map timeframe
            period_map = {"1M": "1d", "5M": "5d", "10M": "5d"}
            period = period_map.get(timeframe, "5d")
```

```
            data = yf.download(ticker, period=period, interval="1m" if timeframe

            if data.empty:
                return None

            return data
        except Exception as e:
            print(f"Error fetching data for {pair}: {e}")
            return None

class TechnicalAnalysis:
    """Calculate technical indicators"""

    @staticmethod
    def calculate_alligator(data, period=13):
        """
        Calculate Alligator Indicator
        Jaw (blue) = SMA(13, high+low)/2, shifted 8 bars
        Teeth (red) = SMA(8, high+low)/2, shifted 5 bars
        Lips (green) = SMA(5, high+low)/2, shifted 3 bars
        """
        if len(data) < period:
            return None, None, None

        hl_avg = (data['High'] + data['Low']) / 2

        jaw = hl_avg.rolling(window=13).mean().shift(8)
        teeth = hl_avg.rolling(window=8).mean().shift(5)
        lips = hl_avg.rolling(window=5).mean().shift(3)

        return jaw.iloc[-1], teeth.iloc[-1], lips.iloc[-1]

    @staticmethod
    def calculate_ema(data, period):
        """Calculate EMA"""
        if len(data) < period:
            return None
        return data['Close'].ewm(span=period, adjust=False).mean().iloc[-1]

    @staticmethod
    def detect_support_resistance(data, lookback=20):
        """Detect support and resistance levels"""
        if len(data) < lookback:
            return None, None

        recent_data = data.tail(lookback)
```

```
        # Support = lowest low
        support = recent_data['Low'].min()

        # Resistance = highest high
        resistance = recent_data['High'].max()

        return support, resistance

    @staticmethod
    def calculate_accuracy(close, jaw, teeth, lips, ema50, ema200, support, resis
        """
        Calculate signal accuracy (0-100%)
        Based on indicator alignment
        """
        accuracy = 0
        checks = 0

        try:
            # 1. Alligator alignment (30 points)
            if jaw is not None and teeth is not None and lips is not None:
                if jaw > teeth > lips:
                    accuracy += 30
                elif teeth > lips > jaw:
                    accuracy += 15
                checks += 1

            # 2. EMA alignment (25 points)
            if ema50 is not None and ema200 is not None:
                if ema50 > ema200:
                    accuracy += 25
                elif ema200 > ema50:
                    accuracy += 12
                checks += 1

            # 3. Price position (20 points)
            if support is not None and resistance is not None:
                if support < close < resistance:
                    accuracy += 20
                checks += 1

            # 4. S&R strength (25 points)
            if support is not None and resistance is not None:
                sr_range = resistance - support
                if sr_range > 0 and sr_range < (close * 0.02):  # Within 2% = str
                    accuracy += 25
                elif sr_range > 0:
                    accuracy += 15
```

```
                checks += 1

            if checks == 0:
                return 0

            # Normalize to 0-100
            accuracy = min(100, (accuracy / (checks * 30)) * 100)

        except Exception as e:
            print(f"Accuracy calculation error: {e}")
            return 0

        return round(accuracy, 1)

class SignalGenerator:
    """Generate trading signals"""

    @staticmethod
    def generate_signal(pair, timeframe, data, accuracy):
        """Generate signal if conditions met"""

        if accuracy < MIN_ACCURACY:
            return None

        if len(data) < 5:
            return None

        close = data['Close'].iloc[-1]
        prev_close = data['Close'].iloc[-2]

        # Determine direction
        if close > prev_close:
            direction = "BUY"
        elif close < prev_close:
            direction = "SELL"
        else:
            return None

        # Calculate entry and targets
        support, resistance = TechnicalAnalysis.detect_support_resistance(data)

        if direction == "BUY":
            entry = close
            sl = support if support else close * 0.99
            target = resistance if resistance else close * 1.01
        else:
            entry = close
```

```
            sl = resistance if resistance else close * 1.01
            target = support if support else close * 0.99

        sl_pips = abs(entry - sl) * 10000  # For JPY pairs
        target_pips = abs(target - entry) * 10000

        if sl_pips == 0:
            sl_pips = 5
        if target_pips == 0:
            target_pips = 25

        rr_ratio = round(target_pips / sl_pips, 2)

        signal = {
            "pair": pair,
            "timeframe": timeframe,
            "direction": direction,
            "entry": round(entry, 5),
            "sl": round(sl, 5),
            "target": round(target, 5),
            "sl_pips": round(sl_pips, 1),
            "target_pips": round(target_pips, 1),
            "rr_ratio": rr_ratio,
            "accuracy": accuracy,
            "time": datetime.now(IST).strftime("%H:%M %Z")
        }

        return signal

async def send_telegram_signal(bot, signal):
    """Send signal to Telegram"""

    emoji_direction = "🟢" if signal["direction"] == "BUY" else "🔴"
    emoji_accuracy = "🔴" if signal["accuracy"] >= 90 else "🟡"

    message = f"""
📊 {signal['pair']} - {signal['direction']} ({signal['timeframe']}) {emoji_direct
━━━━━━━━━━━━━━━━━━━━

⏰ Time: {signal['time']}
📈 Pair: {signal['pair']} | TF: {signal['timeframe']}
📍 Entry: {signal['entry']}
🛑 SL: {signal['sl']} ({signal['sl_pips']} pips)
🎯 Target: {signal['target']} ({signal['target_pips']} pips)
📊 Risk/Reward: 1:{signal['rr_ratio']}

✨ SIGNAL QUALITY:
━━━━━━━━━━━━━━━━━━━━
```

```
{emoji_accuracy} ACCURACY: {signal['accuracy']}% {emoji_accuracy}
✅ High Quality Signal
⭐⭐⭐ CONFIDENCE: VERY HIGH
👉 ACTION: TAKE THIS TRADE
━━━━━━━━━━━━━━━━━━━━━━━━
"""

    try:
        await bot.send_message(chat_id=TELEGRAM_USER_ID, text=message)
        print(f"Signal sent: {signal['pair']} {signal['direction']}")
        return True
    except TelegramError as e:
        print(f"Telegram error: {e}")
        return False

async def check_trading_hours():
    """Check if within trading hours"""
    now = datetime.now(IST)
    current_time = now.strftime("%H:%M")
    day_of_week = now.weekday()  # 0-4 = Mon-Fri

    # Only weekdays
    if day_of_week >= 5:  # Saturday and Sunday
        return False

    # Only 6:30 PM - 8:30 PM IST
    if SIGNAL_TIME_START <= current_time <= SIGNAL_TIME_END:
        return True

    return False

async def analyze_and_send_signals(bot):
    """Main analysis and signal sending function"""

    within_hours = await check_trading_hours()

    if not within_hours:
        return

    fetcher = ForexDataFetcher()
    ta = TechnicalAnalysis()
    sg = SignalGenerator()

    for pair in PAIRS:
        for timeframe in TIMEFRAMES:
            try:
                # Fetch data
```

```
                data = await fetcher.get_price_data(pair, timeframe)

                if data is None or len(data) < 5:
                    continue

                # Calculate indicators
                close = data['Close'].iloc[-1]
                jaw, teeth, lips = ta.calculate_alligator(data)
                ema50 = ta.calculate_ema(data, 50)
                ema200 = ta.calculate_ema(data, 200)
                support, resistance = ta.detect_support_resistance(data)

                # Calculate accuracy
                accuracy = ta.calculate_accuracy(
                    close, jaw, teeth, lips, ema50, ema200, support, resistance
                )

                # Generate signal if accuracy >= 90%
                if accuracy >= MIN_ACCURACY:
                    signal = sg.generate_signal(pair, timeframe, data, accuracy)

                    if signal:
                        # Check if signal already sent (avoid duplicates)
                        signal_key = f"{pair}_{timeframe}_{signal['direction']}"

                        if signal_key not in last_signals or \
                           (datetime.now(IST) - last_signals[signal_key]).seconds

                            await send_telegram_signal(bot, signal)
                            last_signals[signal_key] = datetime.now(IST)
                            signal_count[pair] += 1

                await asyncio.sleep(1)

            except Exception as e:
                print(f"Error analyzing {pair} {timeframe}: {e}")
                continue

async def main():
    """Main bot loop"""

    bot = Bot(token=TELEGRAM_BOT_TOKEN)

    # Send startup message
    try:
        startup_msg = """
🤖 QUOTEX SIGNAL BOT STARTED
```

```
━━━━━━━━━━━━━━━━━━━━━━
✅ Bot: Active
✅ Pairs: EURUSD, GBPUSD, USDJPY
✅ Timeframes: 1M, 5M, 10M
✅ Accuracy Filter: 90%+ ONLY
✅ Time: 6:30 PM - 8:30 PM IST
✅ Days: Weekdays (Mon-Fri)

Ready to send signals! 📊
"""
        await bot.send_message(chat_id=TELEGRAM_USER_ID, text=startup_msg)
    except Exception as e:
        print(f"Startup message error: {e}")

    print("Bot started. Monitoring for signals...")

    # Main loop
    while True:
        try:
            await analyze_and_send_signals(bot)
            await asyncio.sleep(60)  # Check every minute

        except KeyboardInterrupt:
            print("\nBot stopped.")
            break
        except Exception as e:
            print(f"Error in main loop: {e}")
            await asyncio.sleep(60)

if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        print("\nBot stopped by user.")
```